Derate Prediction Modeling

Team **Label-Free**
Nashville Software School
May 21, 2026

Overview

1. Introduction (2 min)
    - The Team
    - Opportunity
    - Solution
2. Project (8 min)
    - Models
    - Challenges
    - Data
    - Targets
    - Actions
    - Projections
3. Outcomes (4 min)
4. Conclusion (1 min)
5. Q & A
6. Appendix

01

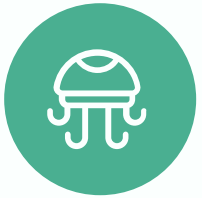
Introduction

- The Team
- Opportunity
- Solution

Team Label-Free

Anitha Doddala

Data Scientist



Shannon Lee

Lead
Data Engineer

Micheal Major

Data Scientist

Josh Tacker

Data Scientist

Opportunity

What is a vehicle derate?

- Vehicle emission system sensors detect certain problematic conditions that will limit engine output. This is known as derate. A full derate or idle level derate is triggered by the fault code SPN 5246.
- Effects:
  - Vehicle will require a tow
  - High cost of repairs
  - Drastically affects customer operations
- Between 2015 - 2018, repairs for full derates cost:

  439 x $4,000 = **$1,776,000**

Solution

Use supervised machine learning models as a tool to predict full derates before they happen.

Ensemble

Random Forest

Gradient Boosting

Neural Network

MLP Classifier

Keras Sequential

02

Project

- Models
- Challenges
- Data
- Targets
- Actions
- Projections

Models

Random Forest

Makes predictions by combining the results of many decision trees, where each tree looks at the data a little differently before “voting” on the final answer. This approach usually produces more reliable and accurate predictions than relying on a single decision tree because it reduces the impact of individual mistakes or overfitting.

Multi-Layer Perceptron (MLP)

Inspired by how the human brain processes information, using layers of connected nodes to learn patterns from data. As it trains, the model adjusts these connections over time to improve its ability to recognize relationships and make predictions on new data.

Gradient Boosting

Builds many small decision trees one after another, with each new tree focusing on correcting the mistakes made by the previous ones. By gradually improving the model step by step, it can make highly accurate predictions, especially for complex patterns in data.

Keras Sequential

A simple way to build a neural network by stacking layers in order, where each layer learns increasingly complex patterns from the data. During training, the model continuously adjusts its internal connections to improve prediction accuracy and adapt to the problem it is solving.

Challenges

Unbalanced Dataset
Over-fitting to the training data

Missing Values
Certain models cannot handle missing values

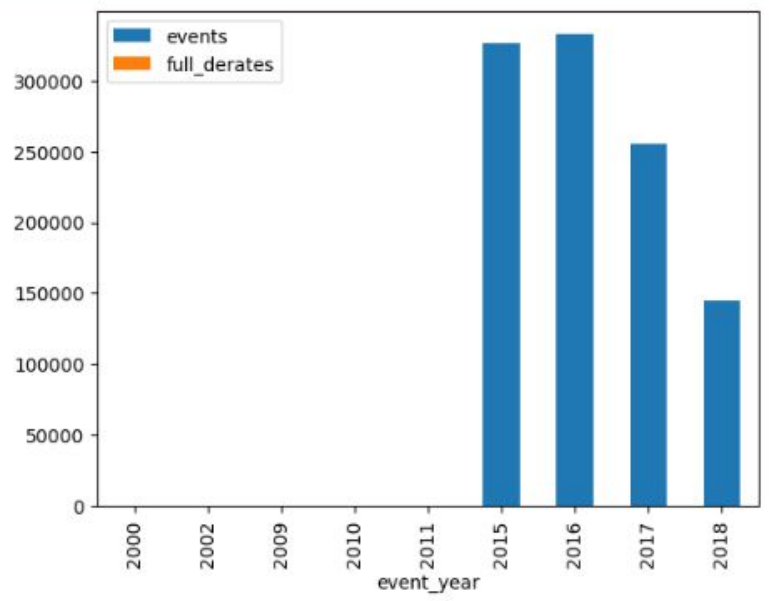
Irrelevant Information
Filter out noise

False Alarms
Servicing can trigger false alarms

Determining a Target
What is the best way to make a prediction

False Positives
Over servicing vehicles

Data



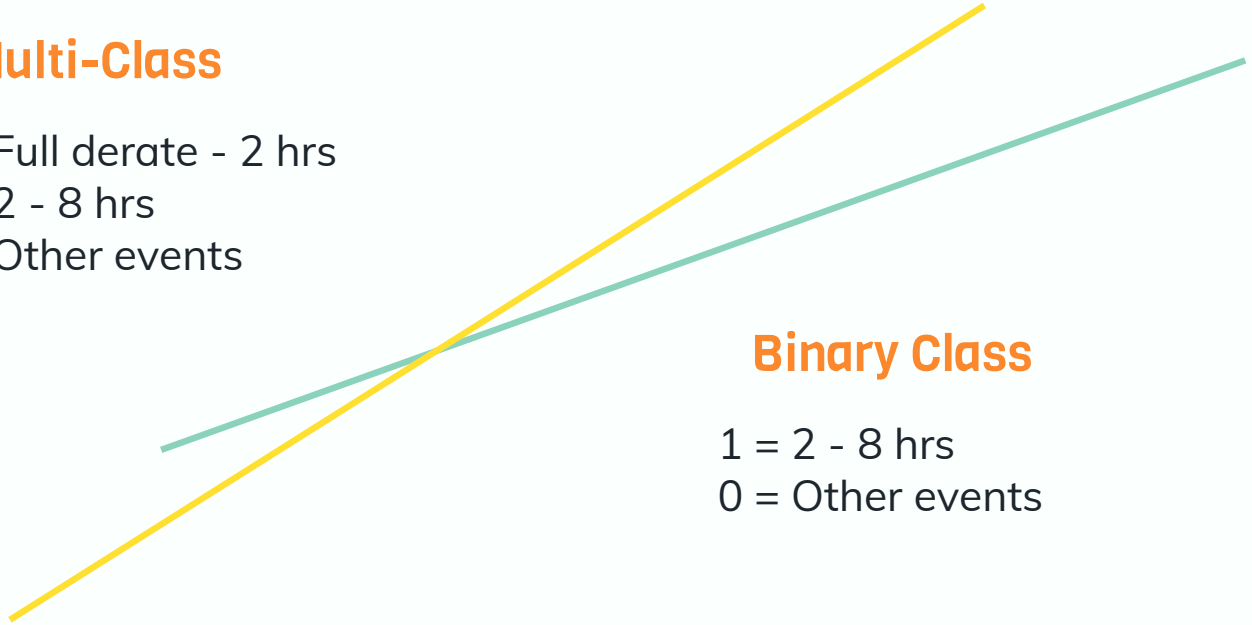
Year	Events	Full Derates	Targets
2000	219	0	0
2002	1	0	0
2009	24	0	0
2010	58	0	0
2011	244	5	0
2015	**325,779**	**68**	**198**
2016	**332,383**	**2,090**	**273**
2017	**254,851**	**81**	**208**
2018	**144,510**	**81**	**209**

Targets

Label by event time windows prior to a full derate

Multi-Class

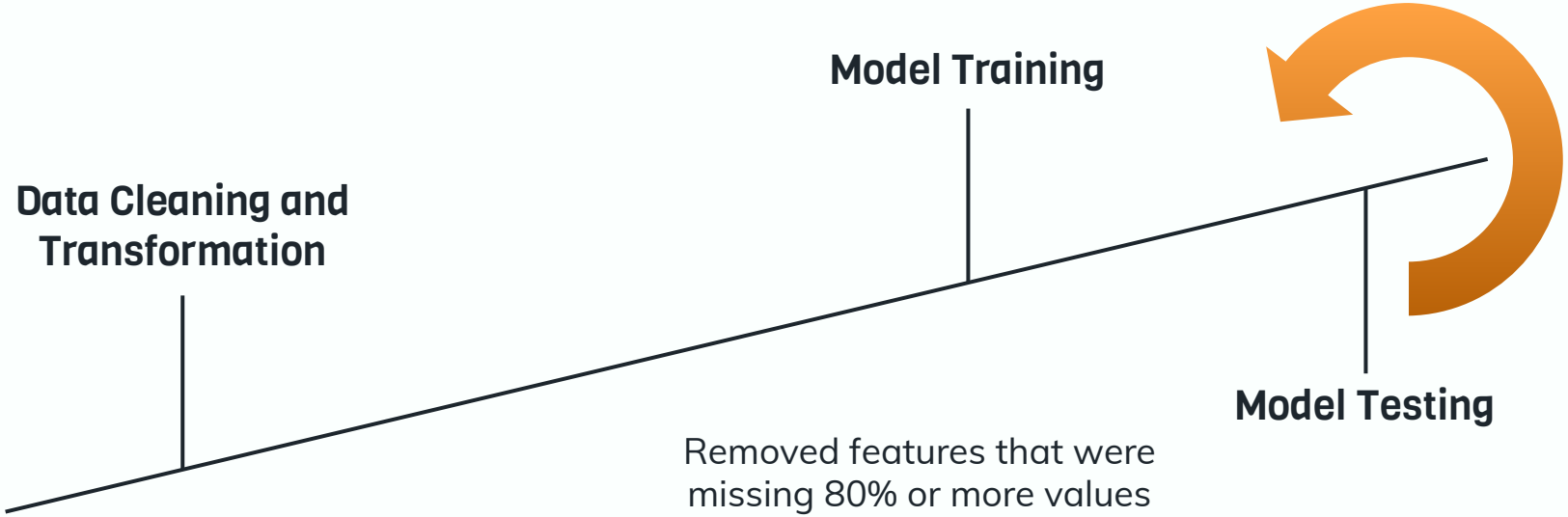
2 = Full derate - 2 hrs
1 = 2 - 8 hrs
0 = Other events



Binary Class

1 = 2 - 8 hrs
0 = Other events

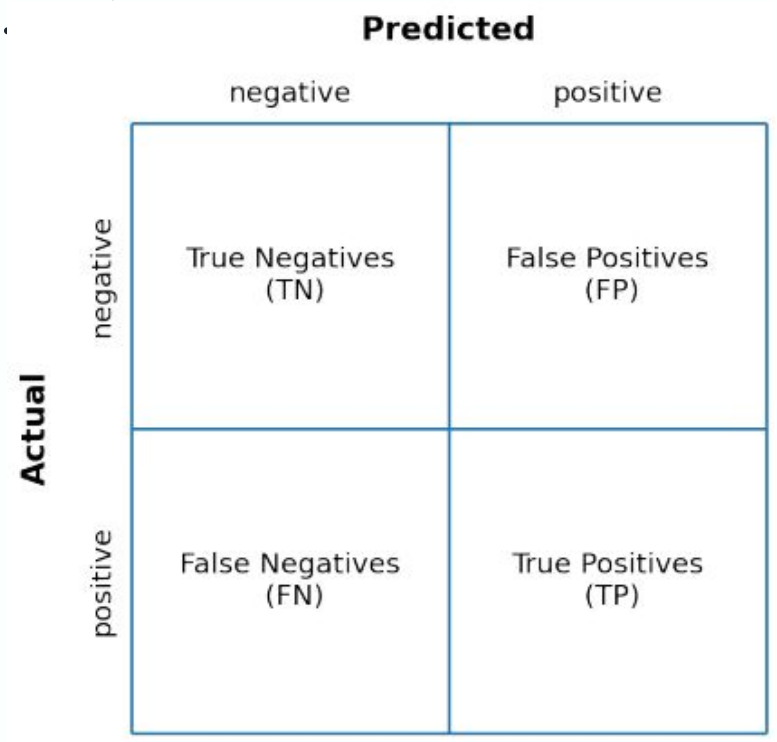
Actions



Identified true derate events and created a target prediction window between 2 - 8 hours prior to a full derate

Removed features that were missing 80% or more values and averaged the values for features missing less than 80% values

Projections



$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Savings = (TP * $4,000) - (FP * $500)

Projections

2000 - 2018

Model	True Positives	False Positives	Savings per 100 Predictions
Multi-Class MLP	**19.4%**	80.6%	$37,300
Multi-Class Keras	**54.4%**	45.6%	$194,800
Binary-Class Keras	**14.0%**	86.0%	$13,000

Projections

2000 - 2018

Model	True Positives	False Positives	Savings
Multi-Class MLP	46	259	$54,500
Multi-Class Keras	17	22	$57,000
Binary-Class Keras	113	733	$13,000

*187 Derate Targets

03

Outcomes

- 2019 - 2020

Outcomes

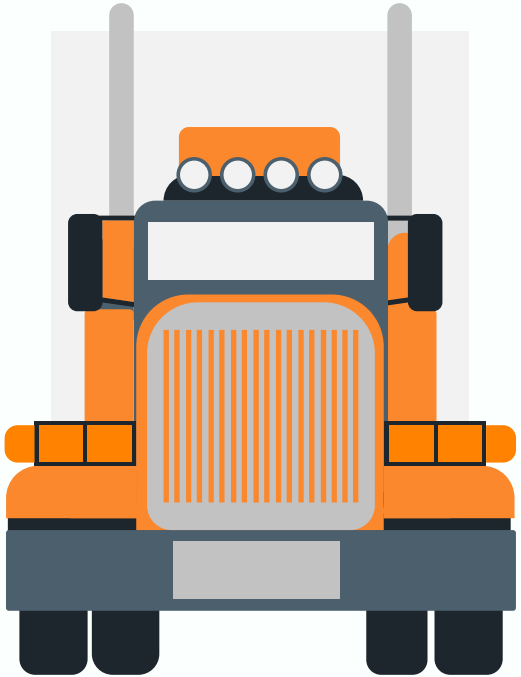
2019 - 2020

Model	True Positives	False Positives	Savings
Multi-Class MLP	30	409	**-$84,500**
Multi-Class Keras	33	764	**-$250,000**
Binary-Class Keras	2	5	$5,500

*40 Derate Targets

04

Conclusion



Conclusion

The project results highlighted several challenges in developing reliable predictive models from the available data. Ensemble models did not achieve sufficient performance to justify continued training, while the multi-class MLPClassifier and Keras Sequential models experienced overfitting, limiting their ability to generalize effectively. Additionally, the binary classification Keras Sequential model generated very few predictions, reducing its practical value for identifying target events.

Conclusion

A significant contributor to these outcomes was the quality and structure of the dataset, particularly the heavy class imbalance and high number of missing values. These limitations made it difficult for the models to learn meaningful patterns across all target classes. External variables not represented in the data, such as newer equipment, vehicle technology advancements, or operational changes, may have also influenced the results.

Conclusion

Despite these challenges, the project provided valuable insight into the complexity of predictive maintenance modeling and identified several opportunities for future improvement, including enhanced data collection, additional feature engineering, and alternative approaches better suited for rare-event prediction.

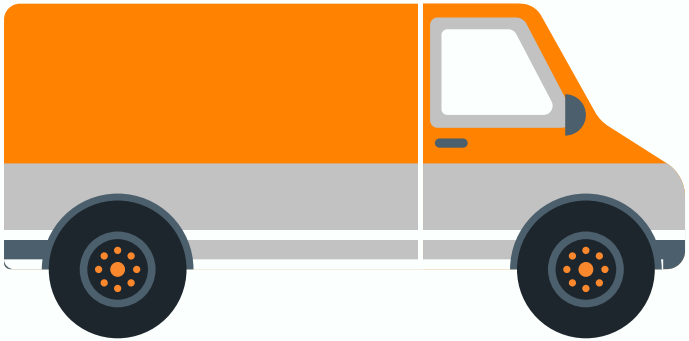
05

Q & A

Thank You

06

Appendix



- Full Dataset Cleaning and Transformation
- Pipeline
- Post-Fit
- Model Training

Full Dataset Cleaning and Transformation

- Converted ‘EventTimeStamp’ and ‘LocationTimeStamp’ to datetime objects
- Converted ‘spn’ and ‘fmi’ to string values
- Using sklearn.neighbors.BallTree, created column ‘NearServiceStation’ to indicate events with 1 mi. of a service station
  - Near: 131367
  - Not Near: 1055968
- Created column IsFullDerate to indicate a full derate: 498 events
  - spn = '5246'
  - active = True
  - NearServiceStation = False
- Using regex, created column ‘Severity_Level’ to indicate code severity level based on ‘eventDescription’
  - Low = 1
  - Medium = 2
  - High = 3
- Created column Derate_Target to indicate the target labels:
  - 2: < 2 hr prior to and including derate (1100)
  - 1: Between 2 - 8 hr prior to full derate (1085)
  - 0: Others (1185150)

Full Dataset Cleaning and Transformation

- Pivot diagnostic dataset wider and merge with faults dataset on RecordID/FaultID
- Replace commas with decimals in numeric features
  - 'AcceleratorPedal',
  - 'BarometricPressure',
  - 'DistanceLtd',
  - 'EngineCoolantTemperature',
  - 'EngineOilPressure',
  - 'EngineOilTemperature',
  - 'EngineRpm',
  - 'EngineTimeLtd',
  - 'FuelLevel',
  - 'FuelLtd',
  - 'FuelRate',
  - 'FuelTemperature',
  - 'IntakeManifoldTemperature',
  - 'Speed',
  - 'SwitchedBatteryVoltage',
  - 'Throttle',
  - 'TurboBoostPressure'
- Split Dataset: Training (1058069 before 2019) / Unseen (129266 after 2019)
- Split Training Chronologically (Bad)

Pipeline

- Set 2 random state variables to get reproducible results and validate ‘lucky’ initializations.
- Split the data for training, validation, and testing
- Impute Missing Values
    - Drop features with > 80% missing values
    - Use SimpleImputer most frequent for categorical features
    - Use SimpleImputer mean for features with < 40% missing values
    - Use IterativeImputer mean for features with > 40% missing values
- One-hot encode non-numeric features
- Scale features
- Adjust the class weight to account for imbalanced dataset
- Set early stopping to prevent overfitting
- Fit model with training dataset

Post-Fit

- Adjust threshold based on a validation dataset f1 score or fbeta score to account for imbalanced classes
- Determine best fit using classification report and confusion matrix
- Calculate Savings / Loss
    - Filter out more than 1 prediction per day

Ensemble: Random Forest

Model: ScikitLlearn Ensemble RandomForestClassifier

This method trains many decision trees using random samples of the data and features so the trees make different, less correlated predictions. The trees are trained independently in parallel, and their predictions are combined by averaging them together.

Target: Binary Derate_Target

Adjust for imbalanced classes:

- Adjust the class weights
- Adjust Threshold

```
Accuracy: 0.9708
Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.97      0.99    264157
           1       0.02      0.49      0.04       361

    accuracy                           0.97    264518
   macro avg       0.51      0.73      0.51    264518
weighted avg       1.00      0.97      0.98    264518

Confusion Matrix:
 [[256605   7552]
 [   184    177]]
```

```
Best Threshold: 0.9099999999999996
Best Net Savings: 106500
```

```
Top 10 Important Features:
                     Feature  Importance
13                       spn    0.427094
14                       fmi    0.129404
0                DistanceLtd    0.064842
10           FuelTemperature    0.058556
12    Severity_Level_Numeric    0.040271
1       EngineOilTemperature    0.037522
5          EngineOilPressure    0.034963
8                  EngineRpm    0.031751
7          BarometricPressure    0.030258
9   IntakeManifoldTemperature    0.028668
```

```
Model Net Savings Results
True Negatives: 256605
False Positives: 7552
False Negatives: 184
True Positives: 12
Money Saved: $-3,728,000.00
```

Ensemble: Gradient Boosting

Model: XGBoost XGBClassifier

Boosting trains decision trees one after another, where each new tree tries to fix the mistakes made by the previous trees. The final prediction is created by combining the trees with weighted contributions, helping improve accuracy while reducing overfitting.

Target: Binary Derate_Target

Adjust for imbalanced classes:

- Adjust the class weights
- Adjust Threshold

```
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00    264157
           1       0.19      0.20      0.19       361

    accuracy                           1.00    264518
   macro avg       0.59      0.60      0.60    264518
weighted avg       1.00      1.00      1.00    264518


Confusion Matrix Results
TP: 71
FP: 309
FN: 290
TN: 263848

Best Threshold: 0.9500000000000001
Best Net Savings: $129,500.00

Money Saved: $129,500.00

Top 10 Important Features:
                     Feature  Importance
13                       spn    0.128330
10           FuelTemperature    0.077621
1        EngineOilTemperature    0.069427
12     Severity_Level_Numeric    0.069110
5          EngineOilPressure    0.068657
4                EngineLoad    0.066728
8                 EngineRpm    0.065742
9   IntakeManifoldTemperature    0.065109
0                DistanceLtd    0.063300
7         BarometricPressure    0.062765
```

Neural Network: MLP Classification

Model: ScikitLearn MLPClassifier (Multi-layer Perceptron classifier)
Learns pattern by making guesses, then improving its guesses by measuring how wrong it was using log-loss.
Target: Multi-Class Derate_Target
Adjust for imbalanced classes:
- Adjust the decision threshold based on f1 score or fbeta score
- Manually split training and testing data chronologically
- Early stopping

Split:
- Train: 0.7 (Validation: 0.1)
- Test: 0.2

Random State: 42, 1
Solver: adam
Activation: relu
Hidden Layers: 32, 32, 32
Early Stopping: 0.1 validation, 2 patience

Neural Network: MLP Classification

Target: Multi-Class Derate_Target
17 Features:

- Categorical:
  - ‘equipmentID’
  - 'spn'
  - 'fmi'
  - 'active'
  - ‘Severity_Level'
- Numeric:
  - 'BarometricPressure'
  - 'EngineCoolantTemperature'
  - 'EngineLoad'
  - 'EngineOilPressure'
  - 'EngineOilTemperature'
  - 'EngineRpm'
  - 'FuelRate'
  - 'FuelTemperature'
  - 'IntakeManifoldTemperature'
  - 'Speed'
  - 'Throttle'
  - ‘TurboBoostPressure'

BEST

Neural Network: MLP Classification

Random State: 42

Training

	precision	recall	f1-score	support
0	0.999144	0.998483	0.998813	950562
1	0.167454	0.488110	0.249361	799
2	0.000000	0.000000	0.000000	901
accuracy			0.997110	952262
macro avg	0.388866	0.495531	0.416058	952262
weighted avg	0.997501	0.997110	0.997240	952262

```
[[949120   1442      0]
 [   409    390      0]
 [   404    497      0]]
```

Testing

	precision	recall	f1-score	support
0	0.999095	0.998443	0.998769	211236
1	0.153101	0.443820	0.227666	178
2	0.000000	0.000000	0.000000	200
accuracy			0.997032	211614
macro avg	0.384065	0.480754	0.408811	211614
weighted avg	0.997439	0.997032	0.997176	211614

```
[[210907    329      0]
 [    99     79      0]
 [    92    108      0]]
```

Unseen

	precision	recall	f1-score	support
0	0.998833	0.988757	0.993769	128970
1	0.056982	0.461929	0.101449	197
2	0.000000	0.000000	0.000000	99
accuracy			0.987197	129266
macro avg	0.351938	0.483562	0.365073	129266
weighted avg	0.996633	0.987197	0.991648	129266

```
[[127520   1450      0]
 [   106     91      0]
 [    43     56      0]]
```

Random State: 1

	precision	recall	f1-score	support
0	0.998855	0.999038	0.998947	950562
1	0.167213	0.319149	0.219449	799
2	0.000000	0.000000	0.000000	901
accuracy			0.997523	952262
macro avg	0.388689	0.439396	0.406132	952262
weighted avg	0.997212	0.997523	0.997347	952262

```
[[949648    914      0]
 [   544    255      0]
 [   545    356      0]]
```

	precision	recall	f1-score	support
0	0.998836	0.998949	0.998892	211236
1	0.158192	0.314607	0.210526	178
2	0.000000	0.000000	0.000000	200
accuracy			0.997429	211614
macro avg	0.385676	0.437852	0.403140	211614
weighted avg	0.997184	0.997429	0.997285	211614

```
[[211014    222      0]
 [   122     56      0]
 [   124     76      0]]
```

	precision	recall	f1-score	support
0	0.998497	0.994231	0.996360	128970
1	0.060213	0.258883	0.097701	197
2	0.000000	0.000000	0.000000	99
accuracy			0.992349	129266
macro avg	0.352903	0.417705	0.364687	129266
weighted avg	0.996302	0.992349	0.994227	129266

```
[[128226    744      0]
 [   146     51      0]
 [    47     52      0]]
```

TP / (TP + FP(0)) = Precision
(79) / (329 + 79) = 19.4%

100 - Precision = FP%
100 - 19.4 = 80.6%

(19.4 * 4000) - (80.6 * 500)
= $37,300 per 100

Neural Network: Keras

Model: TensorFlow Keras Sequential
A simple way to build neural networks by stacking layers one after another
Target: Multi-Class Derate_Target
Split:
    Train: 0.7 (Validation: 0.1)
    Test: 0.2
Random State: 42, 1
Solver: adam
Activation: relu 64, relu 32, softmax 3
Loss: categorical_crossentropy
Metrics: Precision and Recall
Batch Size: 32
Epochs: 20
Early Stopping: 0.1 validation, patience 2
Class Weight: 1, 2, 1
Adjust for imbalanced classes:
    Adjust the decision threshold based on money saved and F-beta score
    Adjust the class weights

Neural Network: Keras

Target: Multi-Class Derate_Target
17 Features:

- Categorical:
  - ‘equipmentID’
  - 'spn'
  - 'fmi'
  - 'active'
  - ‘Severity_Level'
- Numeric:
  - 'BarometricPressure'
  - 'EngineCoolantTemperature'
  - 'EngineLoad'
  - 'EngineOilPressure'
  - 'EngineOilTemperature'
  - 'EngineRpm'
  - 'FuelRate'
  - 'FuelTemperature'
  - 'IntakeManifoldTemperature'
  - 'Speed'
  - 'Throttle'
  - ‘TurboBoostPressure'

BEST

Neural Network: Keras Sequential

Training

	precision	recall	f1-score	support
0	0.998873	0.999803	0.999338	950562
1	0.484536	0.117647	0.189325	799
2	0.710145	0.489456	0.579501	901
micro avg	0.998580	0.998580	0.998580	952262
macro avg	0.731185	0.535635	0.589388	952262
weighted avg	0.998169	0.998580	0.998261	952262
samples avg	0.998580	0.998580	0.998580	952262

```
[[950375     64    123]
 [   648     94     57]
 [   424     36    441]]
```

Testing

	precision	recall	f1-score	support
0	0.998851	0.999839	0.999345	211236
1	0.476190	0.112360	0.181818	178
2	0.708661	0.450000	0.550459	200
micro avg	0.998573	0.998573	0.998573	211614
macro avg	0.727901	0.520733	0.577207	211614
weighted avg	0.998137	0.998573	0.998233	211614
samples avg	0.998573	0.998573	0.998573	211614

```
[[211202     14     20]
 [   141     20     17]
 [   102      8     90]]
```

Unseen

	precision	recall	f1-score	support
0	0.998820	0.984454	0.991585	128970
1	0.039921	0.411168	0.072776	197
2	0.081967	0.101010	0.090498	99
micro avg	0.982903	0.982903	0.982903	129266
macro avg	0.373569	0.498877	0.384953	129266
weighted avg	0.996656	0.982903	0.989494	129266
samples avg	0.982903	0.982903	0.982903	129266

```
[[126965   1895    110]
 [   114     81      2]
 [    36     53     10]]
```

Random State: 1

	precision	recall	f1-score	support
0	0.998810	0.999866	0.999338	950562
1	0.412587	0.073842	0.125265	799
2	0.777174	0.476138	0.590502	901
micro avg	0.998594	0.998594	0.998594	952262
macro avg	0.729524	0.516615	0.571702	952262
weighted avg	0.998109	0.998594	0.998218	952262
samples avg	0.998594	0.998594	0.998594	952262

```
[[950435     61     66]
 [   683     59     57]
 [   449     23    429]]
```

	precision	recall	f1-score	support
0	0.998799	0.999867	0.999333	211236
1	0.350000	0.078652	0.128440	178
2	0.803571	0.450000	0.576923	200
micro avg	0.998573	0.998573	0.998573	211614
macro avg	0.717457	0.509506	0.568232	211614
weighted avg	0.998069	0.998573	0.998201	211614
samples avg	0.998573	0.998573	0.998573	211614

```
[[211208     18     10]
 [   152     14     12]
 [   102      8     90]]
```

	precision	recall	f1-score	support
0	0.998245	0.996464	0.997354	128970
1	0.039106	0.071066	0.050450	197
2	0.279762	0.474747	0.352060	99
micro avg	0.994654	0.994654	0.994654	129266
macro avg	0.439038	0.514093	0.466621	129266
weighted avg	0.996233	0.994654	0.995416	129266
samples avg	0.994654	0.994654	0.994654	129266

```
[[128514    342    114]
 [   176     14      7]
 [    50      2     47]]
```

TP / (TP + FP(0)) = Precision
(20) / (20 + 14) = 58.8%

100 - Precision = FP%
100 - 58.8 = 42.2%

(58.8 * 4000) - (42.2 * 500)
= $37,300 per 100

Neural Network: Keras

Best model:

Target: 'Derate_Target_8.0-2.0'

Split:

Train: 0.7 (train-75%, val-25%)
Test: 0.3

Activation:
Dense Layer : relu 64 Neurons+ReLU with 20% dropout
Dense Layer : relu 32 Neurons+ReLU with 20% dropout

Output: Sigmoid activation

Handled Imbalanced Data:

```
class_weights = {
    0: 1,
    1: 30
}
```

Equipment and Daily Aggregation

- Multiple records existed for each equipment
- Predictions aggregated by:
  a. EquipmentID
  b. Event Date

And we used

- sum of predictions to count warning signals
- max of target to determine actual failure occurrence”

Neural Network: Keras Sequential

Training

	precision	recall	f1-score	support
0	1.00	1.00	1.00	555019
1	0.19	0.55	0.28	466
accuracy			1.00	555485
macro avg	0.59	0.78	0.64	555485
weighted avg	1.00	1.00	1.00	555485

```
[[553921   1098]
 [   208    258]]
```

Testing

	precision	recall	f1-score	support
0	1.00	1.00	1.00	317155
1	0.14	0.42	0.21	266
accuracy			1.00	317421
macro avg	0.57	0.71	0.60	317421
weighted avg	1.00	1.00	1.00	317421

```
[[316441    714]
 [   153    113]]
```

Unseen

	precision	recall	f1-score	support
0	1.00	1.00	1.00	24459
1	0.29	0.05	0.09	40
accuracy			1.00	24499
macro avg	0.64	0.52	0.54	24499
weighted avg	1.00	1.00	1.00	24499

```
[[24454     5]
 [   38     2]]
```

Validation -Threshold-0.74

	precision	recall	f1-score	support
0	1.00	1.00	1.00	317155
1	0.38	0.19	0.25	266
accuracy			1.00	317421
macro avg	0.69	0.59	0.63	317421
weighted avg	1.00	1.00	1.00	317421

```
[[317074     81]
 [   216     50]]
```

TP / (TP + FP(0)) = Precision
(113) / (714+ 113) =14%

100 - Precision = FP%
100 - 14 = 86%

(14 * 4000) - (86 * 500) = $13000 per 100